
Massive Data Fundamentals In One Question

*Why does a computer look the way it does
and when does that design break down?*

Part 1

The Bet on CPUs

The Problem Before CPUs



The Rewiring Problem

ENIAC (1945) computed at electronic speed, but changing the program meant **physically rewiring the machine.**

Up to 3 weeks per new program.

Every task needed its own circuit.

That doesn't scale.



The Breakthrough

Von Neumann, Eckert & Mauchly: store **instructions and data in the same memory.**

One flexible circuit interprets instructions sequentially.

The machine becomes reprogrammable without physical modification.

EDVAC Report, 1945

The CPU is a bet: flexibility is more valuable than raw speed for most workloads.

When the Bet Breaks Down

When a specific operation becomes the binding constraint across enough workloads, the economics flip.

GPUs

Graphics needed massive parallel floating-point throughput that CPUs couldn't deliver sequentially.

TPUs

Google built custom silicon because matrix multiplication at scale justified the fixed cost.

FPGAs / ASICs

When the workload is well-defined and high-volume, even reprogrammability is overhead.

Goldratt's “**elevate the constraint**” (applied to silicon)

The boundary between general and specialized isn't fixed — it moves as workloads evolve.

Part 2

CERN: All Three Paradigms

The Scale of the Problem

300

GB/s

Raw data from
LHC detectors

170+

sites

Computing centers
across 42 countries

1.4M

cores

CPU cores in the
Worldwide LHC Grid

50-100×

More capacity needed
for HL-LHC (~2029)

No single computing paradigm can handle this. Different parts of the problem have different binding constraints.

LAYER 1

Specialized Edge Computing

Binding constraint: I/O bandwidth and storage. You physically cannot store or transmit 300 GB/s.

The Trigger System: 500:1 Data Reduction

Custom hardware triggers decide in $<10 \mu\text{s}$ which collisions to keep

LHCb HLT1 (GPUs): 5 TB/s $\rightarrow$ 120 GB/s (keep ~3%)

LHCb HLT2 (CPUs): 120 GB/s $\rightarrow$ 10 GB/s (final storage)

Why general-purpose fails: Latency. Decisions must be synchronized with beam crossings at microsecond precision. A CPU fetching and decoding instructions cannot meet this timing.

Policy parallel: Sensor networks, military surveillance, autonomous vehicles — anywhere data volume at source overwhelms transmission.

LAYER 2

HPC: Tightly Coupled Computation

Binding constraint: Inter-processor communication speed. Each step depends on the previous one. Network latency kills performance.

HPC Clusters with Fast Interconnects

Event reconstruction and Monte Carlo simulation require tightly coupled processing

ALICE: 250 nodes × (8 GPUs + 2×32-core CPUs)
Processing 600 GB/s of partially filtered data

Why distributed computing fails: Sending intermediate results across the internet between every computation step makes real-time reconstruction impossible. Processors need physical proximity.

Policy parallel: Weather forecasting requires constant data exchange between grid cells requires tightly coupled HPC.

LAYER 3

Distributed Computing: The WLCG

Binding constraint: Aggregate compute capacity and cost. No single institution can house enough computing for 200 PB/year.

The Worldwide LHC Computing Grid

Tier 0 (CERN): Initial reconstruction + backup

Tier 1 (13 major centers): Reprocessing specific subsets

Tier 2 (150+ sites): Specific physics analyses

Individual physicists: Local clusters and laptops

Why HPC fails: The analysis jobs are embarrassingly parallel. Each physicist analyzes a different data subset independently. No inter-processor communication needed. Commodity hardware worldwide is far more cost-effective.

Policy parallel: Federated data analysis: Census, multi-agency datasets, any workload decomposable into independent tasks.

Three Paradigms, One System

	Edge / Specialized	HPC / Tightly Coupled	Distributed / Loosely Coupled
Constraint	Latency, I/O at source	Communication speed	Capacity & cost
Hardware	Custom ASICs, FPGAs, GPUs	Supercomputers, InfiniBand	Commodity servers
Data strategy	Process where generated	Fast local interconnect	Partition to avoid movement
Workload	Real-time filtering	Simulation, reconstruction	Embarrassingly parallel
CERN example	Trigger: 40 MHz → 1 MHz	ALICE GPU farm	WLCG: 170 sites, 42 countries
Gov't parallel	IoT, military edge, AV	Weather, nuclear sim	Census, federated analysis

CERN uses all three simultaneously, because different parts of the problem have different binding constraints.

Part 3

The Semester in Retrospect

Everything Maps to This Framework

Week 4

Asked “what’s the binding constraint?” at single-computer scale. Today: the same question at planetary scale.

Week 6

Glue/Athena pipeline: process data where it sits. Same principle as WLCG Tier 2 analysis.

Week 7

RDS, Redshift, Lake Formation each exist because a general-purpose approach hit a constraint.

Weeks 9–12

Spark’s model (partition, process independently, combine) is exactly what the WLCG does across 42 countries.

The Question You Should Always Ask

“What is the binding constraint in this system, and is the proposed solution addressing that constraint — or a different one?”

We started the semester accepting the overhead of general-purpose computing because flexibility matters. We end it understanding when that assumption breaks down — and what replaces it.

PPOL 5206

Massive Data Fundamentals

Georgetown McCourt School of Public Policy · Spring 2026